PSI 2025-2026 – Objectif 3 :
Base de données & Persistance

Ce document guide la réalisation de l'**Objectif 3** : implémenter la **persistance des graphes et tournées** dans une base de données MySQL, à travers une classe ServicePersistance en C#.

Prérequis : avoir terminé les objectifs 1 (type Graphe / Matrix) et 2 (algorithme de Little / Tour).

Éléments fournis

Pour vous aider dans cet objectif, les éléments suivants vous sont fournis :

- Classe ServicePersistance : patron à compléter pour la persistance en base de données
- Script SQL init_db.sql : structure de base pour initialiser votre base de données (à compléter)
- Classe PersistanceTests : tests d'intégration pour valider le bon fonctionnement du service de persistance

Pour intégrer ces éléments à votre projet,

Un des membres du groupe doit réaliser les opérations suivantes :

1. Récupérer le fichier ZIP contenant ces éléments sur DVL.
2. Dézipper le fichier.
3. Ouvrir votre projet PSI dans Visual Studio.
4. Glisser-déposer ServicePersistance.cs dans le projet « TourneeFutee ».
5. Glisser-déposer PersistanceTests.cs dans le projet « TourneeFutee.Tests ».
6. Lancer les tests de la campagne « PersistanceTests » : tout doit compiler, les tests doivent se lancer et échouer.

Puis, pour que les autres membres du groupe puissent en profiter :

1. Valider les modifications dans votre dépôt local (*commit*)
2. Pousser les modifications sur le dépôt distant (*push*)
3. Demander aux autres membres du groupe de tirer les modifications (*pull*)

À présent, tous les membres ont un projet à jour en local.

Partie 1 : Conception de la base de données

Rappel du contexte

Le service de persistance doit permettre de stocker et recharger :

- Des **graphes** : leurs sommets (avec valeurs) et leurs arcs/arêtes (avec poids)
- Des **tournées** : la séquence de sommets visités et le coût total, rattachée au graphe correspondant

Identification des entités

Avant de modéliser le diagramme Entité/Association (E/A), identifiez les entités à stocker.

Question 1 : Quelles entités devez-vous représenter en base de données pour stocker un graphe complet (sommets + arcs) ? Listez-les et justifiez.

Question 2 : Quelles informations supplémentaires faut-il stocker pour représenter une tournée ? Pensez à l'ordre des étapes et au coût total.

Question 3 : Quelles relations existent entre ces entités ? Précisez les cardinalités (1,1 / 1,n / 0,n, etc.).

Modélisation E/A

Réalisez le **diagramme Entité/Association** modélisant votre base de données. Il sera intégré au rapport final.

Vérifiez que votre modèle couvre au minimum les éléments suivants :

Entité / Relation	Attributs minimaux attendus	Remarques
Graphe	Identifiant (PK), nombre de sommets	Généré automatiquement (AUTO_INCREMENT)
Sommet	Identifiant (PK), valeur, clé étrangère vers Graphe	Un graphe contient plusieurs sommets
Arc / Arête	Identifiant (PK), sommet source (FK), sommet destination (FK), poids, clé vers Graphe	Attention : graphe orienté vs non orienté
Tournée	Identifiant (PK), coût total, clé étrangère vers Graphe	Une tournée est liée à un graphe précis
Étape de tournée	Clé vers Tournée (FK), numéro d'ordre, clé vers Sommet (FK)	Permet de reconstruire la séquence ordonnée

Implémentation SQL

Complétez le script SQL init_db.sql fourni pour créer toutes les tables avec leurs contraintes (clés primaires, clés étrangères, types adaptés).

Conseils :

– Utilisez INT UNSIGNED AUTO_INCREMENT pour les identifiants générés automatiquement.

– Déclarez les clés étrangères avec FOREIGN KEY ... REFERENCES ... et pensez à ON DELETE CASCADE si nécessaire.

– Choisissez des types adaptés : VARCHAR pour les noms de sommets, FLOAT ou DOUBLE pour les poids.

Partie 2 : Rappels – Interactions avec une BdD en C#

Vous avez déjà utilisé MySQL depuis un programme C# lors du cours de Bases de Données de S1. Cette partie rappelle les éléments essentiels.

Les 4 étapes clés

Étape	Classe C#	Description
1. Connexion	MySqlConnection	Créer et ouvrir une connexion avec la chaîne de connexion (serveur, BdD, user, mdp)
2. Requête	MySqlCommand	Créer la commande SQL avec des paramètres (éviter les injections !)
3. Exécution	ExecuteReader() / ExecuteNonQuery() / ExecuteScalar()	SELECT → Reader | INSERT/UPDATE/DELETE → NonQuery | valeur unique → Scalar
4. Parcours	MySqlDataReader	Itérer avec reader.Read(), lire les colonnes avec reader["col"] ou reader.GetInt32(0)

Exemple de code : insérer un sommet

Voici un exemple d'insertion d'un sommet en base de données depuis C# :

```
string connStr = $"server={serverIp};database={dbname};uid={user};pwd={pwd}";
using (var conn = new MySqlConnection(connStr))
{
    conn.Open();
    string sql = "INSERT INTO Sommet(valeur, graphe_id) VALUES (@v, @gid);
SELECT LAST_INSERT_ID();";
```

```
    var cmd = new MySqlCommand(sql, conn);
    cmd.Parameters.AddWithValue("@v", sommet.Valeur);
    cmd.Parameters.AddWithValue("@gid", grapheId);
    uint id = Convert.ToUInt32(cmd.ExecuteScalar());
}
```

Points importants :

– Toujours utiliser des paramètres (@v, @gid) : cela prévient les injections SQL.

– Le mot-clé using garantit la fermeture de la connexion même en cas d'exception.

– LAST_INSERT_ID() retourne l'identifiant généré par AUTO_INCREMENT après un INSERT.

– Pour charger des données, utiliser ExecuteReader() puis boucler avec reader.Read().

Partie 3 : Implémentation du service de persistance

La classe ServicePersistance

En C#, le service de persistance est modélisé par la classe ServicePersistance fournie, dont le code est à compléter. Voici l'interface à implémenter :

Méthode	Description
ServicePersistance(string serverIp, string dbname, string user, string pwd)	Instancie le service et se connecte automatiquement à la base de données dbname sur le serveur serverIp, avec les identifiants user et pwd.
unsigned int SaveGraph(Graph g)	Sauvegarde le graphe g en base de données (sommets + arcs) et retourne son identifiant.
Graph LoadGraph(unsigned int id)	Charge depuis la base le graphe identifié par id et retourne une instance de la classe Graph.
unsigned int SaveTour(unsigned int graphId, Tour t)	Sauvegarde la tournée t (effectuée dans le graphe graphId) en base de données et retourne son identifiant.
Tour LoadTour(unsigned int id)	Charge depuis la base la tournée identifiée par id et retourne une instance de la classe Tour.

Ordre d'implémentation conseillé

Nous vous conseillons d'implémenter les méthodes dans l'ordre suivant :

Étape	Méthode à implémenter	Test d'intégration associé
1	Constructeur (connexion à la BdD)	PersistanceTests.ConnectionTest
2	SaveGraph(Graph g)	PersistanceTests.SaveGraphTest
3	LoadGraph(uint id)	PersistanceTests.SaveAndLoadGraphTest
4	SaveTour(uint graphId, Tour t)	PersistanceTests.SaveTourTest
5	LoadTour(uint id)	PersistanceTests.SaveAndLoadTourTest

Conseils d'implémentation

SaveGraph : sauvegarder dans le bon ordre

Respectez l'ordre suivant pour éviter les violations de clés étrangères :

- Insérer d'abord une ligne dans la table Graphe → récupérer son identifiant
- Ensuite, pour chaque sommet : insérer dans Sommet avec la clé étrangère vers ce graphe
- Enfin, pour chaque arc de la matrice d'adjacence : insérer dans Arc avec les clés étrangères vers les sommets

LoadGraph : reconstruire le graphe en C#

- Charger les informations du graphe (orienté ou non) depuis la table Graphe
- Charger tous les sommets liés à ce graphe depuis Sommet : reconstruire la liste dans le même ordre qu'à la sauvegarde
- Charger tous les arcs depuis Arc : reconstituer la matrice d'adjacence

SaveTour / LoadTour : conserver l'ordre des étapes

- Insérer une ligne dans Tournée avec le coût total et la clé vers le graphe
- Pour chaque sommet de la séquence : insérer dans EtapeTournée avec son numéro d'ordre
- Au chargement : trier les étapes par numéro d'ordre pour reconstruire la séquence correcte

Erreurs fréquentes à éviter :

– Oublier de sauvegarder les sommets AVANT les arcs (violation de clé étrangère)

– Concaténer des valeurs dans le SQL au lieu d'utiliser des paramètres

– Ne pas gérer les exceptions SQL (entourer les appels BdD d'un bloc try/catch)

– Ne pas fermer la connexion après utilisation (utiliser using)

Partie 4 : Tests d'intégration

Les tests de la campagne PersistanceTests vérifient que votre service de persistance fonctionne correctement de bout en bout.

Stratégie de test

Pour chaque méthode, le principe des tests est le suivant :

- Sauvegarder un objet (graphe ou tournée) → récupérer son identifiant
- Recharger l'objet depuis la base avec cet identifiant
- Vérifier que l'objet rechargé est identique à l'original (mêmes sommets, mêmes poids, même ordre des étapes)

Cas de test à valider

Test	Ce qui est vérifié
ConnectionTest	La connexion à la BdD s'établit sans exception
SaveGraphTest	SaveGraph retourne un identifiant valide (> 0)
SaveAndLoadGraphTest	Le graphe rechargé est identique au graphe sauvegardé (sommets, arcs, poids)
SaveTourTest	SaveTour retourne un identifiant valide (> 0)
SaveAndLoadTourTest	La tournée rechargée est identique (même séquence, même coût)
MultipleGraphsTest	Plusieurs graphes peuvent coexister en base sans interférence

Conseil : Utilisez une base de données de test dédiée (ex : tourneefutee_test) distincte de votre base principale. Vous pouvez vider les tables entre deux séries de tests pour garantir l'indépendance des cas.

Mise à jour du diagramme de classes

Mettez à jour le diagramme de classes UML démarré dans l'objectif 1 (et mis à jour dans l'objectif 2) en ajoutant :

- La classe ServicePersistance et ses méthodes publiques
- Les associations entre ServicePersistance, Graph et Tour
- Les éventuelles classes utilitaires que vous avez ajoutées

Cette mise à jour sera évaluée dans le rapport final (Objectif 4).

Rendu

Soumissions régulières : sur GitHub

Tous les membres doivent contribuer au dépôt. Les commits de chaque membre permettront notamment à votre encadrant d'apprécier votre apport au projet.

Dépôt final pour évaluation : zip sur DVL

Le travail évalué sera votre dépôt sur DVL.

Pour rendre votre travail sur DVL :

7. Allez sur la page GitHub de votre dépôt.
8. Cliquez sur « Code » puis « Download zip ».
9. Renommez votre fichier zip en psi_objectif3_groupeX.zip, où X est votre nom de groupe.
10. Déposez votre fichier zip sur DVL, activité « Rendu objectif 3 », en utilisant le compte du **représentant du groupe** (ne faire qu'un seul dépôt).

Date limite de rendu : à préciser par votre encadrant

Pénalités de retard : 1 point par heure de dépassement de la deadline.

Évaluation

L'objectif 3 est évalué sur les tests d'intégration *et* sur le rapport final.

Type	Élément	Poids	Moyen d'évaluation
Évaluation de groupe (75 %)	Code – Persistance en BdD	15 %	Tests d'intégration (PersistanceTests)
	Rapport – Diagramme E/A	10 %	Relecture par encadrant
	Rapport – Diagramme de classes (mise à jour)	8 %	Relecture par encadrant
	Rapport – Script SQL	7 %	Relecture par encadrant
Évaluation individuelle (25 %)	Validation des connaissances (quiz 4/4)	15 %	Quiz en ligne (5 questions, 5 min)
	Revue de code	10 %	Questions orales (en TD)

Si le code ne compile pas ⇒ Note = 0

Vous connaissez donc votre note de tests avant de rendre votre travail.